



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Trabajo Práctico N° 2

Introducción a la Estructura de Datos

Alumno: Facundo Luna

Profesor: Ing. Carlos Pastrana

Fecha: 5 de mayo de 2026

Índice

1. Algoritmo con pila — cadena inversa	2
2. Algoritmos sobre listas linkeadas	6
3. Planilla de cálculo con listas linkeadas	13

1. Escribir la secuencia de pasos básicos de un algoritmo, para determinar con el uso de una pila si una cadena x de entrada es válida, tal que conste de las letras 'A' y 'B', y genere una cadena inversa w de salida.

Solución:

Descripción del algoritmo

El algoritmo debe validar que la cadena de entrada X contenga únicamente los caracteres 'A' y 'B'. Posteriormente, utiliza una **pila** (estructura LIFO — *Last In First Out*) para generar la cadena inversa W .

Pasos del algoritmo:

1. **Validación:** Recorrer cada carácter de X verificando que sea 'A' o 'B'. Si se encuentra un carácter inválido, retornar error.
2. **Apilado:** Una vez validada la cadena, recorrerla nuevamente apilando cada carácter en orden de aparición.
3. **Desapilado:** Construir la cadena inversa W desapilando todos los caracteres. Debido a la naturaleza LIFO de la pila, los caracteres salen en orden inverso al que fueron insertados.

Pseudocódigo

```
1  ALGORITMO generarCadenaInversa(X)
2      // Validacion
3      PARA CADA caracter c EN X HACER
4          SI c != 'A' Y c != 'B' ENTONCES
5              RETORNAR ERROR
6          FIN_SI
7      FIN_PARA
8
9      // Apilar caracteres
10     CREAR pila
11     PARA CADA caracter c EN X HACER
12         pila.PUSH(c)
13     FIN_PARA
14
15     // Desapilar para formar W
16     W = cadena vacia
17     MIENTRAS pila NO este vacia HACER
18         W = W + pila.POP()
19     FIN_MIENTRAS
20
21     RETORNAR W
22 FIN_ALGORITMO
```

Implementación en Java

```
1 import java.util.Stack;
2
3 public class ReverseString {
4
5     public static String reverseWithStack(String input) {
6         // Validacion: solo 'A' y 'B'
7         for (int i = 0; i < input.length(); i++) {
8             char c = input.charAt(i);
9             if (c != 'A' && c != 'B') {
10                 return null; // Cadena invalida
11             }
12         }
13
14         // Apilar todos los caracteres
15         Stack<Character> stack = new Stack<>();
16         for (int i = 0; i < input.length(); i++) {
17             stack.push(input.charAt(i));
18         }
19
20         // Desapilar para formar la cadena inversa
21         StringBuilder reversed = new StringBuilder();
22         while (!stack.isEmpty()) {
23             reversed.append(stack.pop());
24         }
25
26         return reversed.toString();
27     }
28 }
```

Ejemplo de ejecución

Entrada (X)	Salida (W)
AABBA	ABBAA
BABA	ABAB
AAA	AAA
AB	BA

Análisis de complejidad

- **Complejidad temporal:** $O(n)$
 - Validación: $O(n)$ — un recorrido completo
 - Apilado: $O(n)$ — inserción de n elementos
 - Desapilado: $O(n)$ — extracción de n elementos
 - Total: $O(n) + O(n) + O(n) = O(n)$
- **Complejidad espacial:** $O(n)$
 - Pila: almacena los n caracteres de la cadena
 - Cadena resultado: $O(n)$
 - Total: $O(n)$

[↑ Volver al índice](#)

2. Escribir la secuencia de pasos básicos de un algoritmo para:

- Concatenar dos listas linkeadas.
- Invertir una lista, de tal manera que el último elemento pase a ser el primero, y así sucesivamente.
- Retornar la suma de los números enteros que contiene los elementos de una lista.

Solución:

Estructura del nodo

Todos los algoritmos asumen la siguiente definición de nodo:

```
1 class Node {  
2     int value;          // Dato almacenado  
3     Node next;         // Referencia al siguiente nodo  
4  
5     public Node(int value) {  
6         this.value = value;  
7         this.next = null;  
8     }  
9 }
```

a) Concatenar dos listas linkeadas

Descripción: El algoritmo recorre la primera lista hasta encontrar el último nodo (aquel cuyo campo `next` es `null`), y enlaza ese nodo con el primer nodo de la segunda lista.

Casos especiales:

- Si `list1` es `null`, retornar `list2`
- Si `list2` es `null`, retornar `list1`

Pseudocódigo

```
1 ALGORITMO concatenar(list1, list2)
2   // Casos especiales
3   SI list1 == NULO ENTONCES
4     RETORNAR list2
5   FIN_SI
6   SI list2 == NULO ENTONCES
7     RETORNAR list1
8   FIN_SI
9
10  // Recorrer hasta el ultimo nodo de list1
11  current = list1
12  MIENTRAS current.next != NULO HACER
13    current = current.next
14  FIN_MIENTRAS
15
16  // Enlazar con list2
17  current.next = list2
18
19  RETORNAR list1
20 FIN_ALGORITMO
```

Implementación en Java

```
1 public static Node concatenate(Node list1, Node list2) {
2   // Casos especiales:
3   if (list1 == null) {
4     return list2;
5   }
6   if (list2 == null) {
7     return list1;
8   }
9
10  // Buscar el ultimo nodo de list1
11  Node current = list1;
12  while (current.next != null) {
13    current = current.next;
14  }
15
16  current.next = list2;
17
18  return list1;
19 }
```


Complejidad:

- **Temporal:** $O(n)$ donde n es la longitud de la primera lista
- **Espacial:** $O(1)$ — no se crean nodos nuevos, solo se modifican referencias

b) Invertir una lista linkeada

Descripción: El algoritmo invierte los enlaces de la lista *in-place* (sin crear nodos nuevos). Utiliza tres referencias que avanzan simultáneamente:

- **previous:** apunta al nodo anterior (inicialmente null)
- **current:** apunta al nodo actual que se está procesando
- **next:** guarda temporalmente el siguiente nodo antes de modificar el enlace

Idea clave: En cada iteración, se invierte el enlace del nodo actual para que apunte hacia atrás, luego se avanzan las tres referencias.

Pseudocódigo

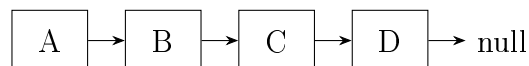
```
1  ALGORITMO invertir(head)
2      previous = NULO
3      current = head
4
5      MIENTRAS current != NULO HACER
6          // Guardar referencia al siguiente nodo
7          next = current.next
8
9          // Invertir el enlace
10         current.next = previous
11
12         // Avanzar las referencias
13         previous = current
14         current = next
15     FIN_MIENTRAS
16
17     // 'previous' es ahora el nuevo primer nodo
18     RETORNAR previous
19 FIN_ALGORITMO
```

Implementación en Java

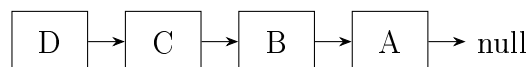
```
1 public static Node reverse(Node head) {  
2     Node previous = null;  
3     Node current = head;  
4     Node next = null;  
5  
6     while (current != null) {  
7         // Guardar el siguiente nodo  
8         next = current.next;  
9  
10        // Invertir el enlace  
11        current.next = previous;  
12  
13        // Avanzar las referencias  
14        previous = current;  
15        current = next;  
16    }  
17  
18    // 'previous' ahora apunta al nuevo primer nodo  
19    return previous;  
20 }
```

Visualización del proceso:

Original



Invertida



Complejidad:

- **Temporal:** $O(n)$ — un solo recorrido de la lista
- **Espacial:** $O(1)$ — solo se usan tres referencias auxiliares

c) Suma de números enteros en una lista

Descripción: El algoritmo recorre toda la lista acumulando el valor entero de cada nodo en una variable sum.

Pseudocódigo

```
1 ALGORITMO sumarLista(head)
2   sum = 0
3   current = head
4
5   MIENTRAS current != NULO HACER
6       sum = sum + current.value
7       current = current.next
8   FIN_MIENTRAS
9
10  RETORNAR sum
11 FIN_ALGORITMO
```

Implementación en Java

```
1 public static int sumList(Node head) {
2     int sum = 0;
3     Node current = head;
4
5     while (current != null) {
6         sum += current.value;
7         current = current.next;
8     }
9
10    return sum;
11 }
```

Ejemplo:

Si la lista contiene: $5 \rightarrow 3 \rightarrow 8 \rightarrow 2 \rightarrow \text{null}$

El resultado será: $5 + 3 + 8 + 2 = 18$

Complejidad:

- **Temporal:** $O(n)$ — se visita cada nodo exactamente una vez
- **Espacial:** $O(1)$ — solo se usa una variable acumuladora

[↑ Volver al índice](#)

3. Indicar cómo representar, con el uso de listas lineadas, una planilla de cálculo electrónica de 999×999 celdas. ¿Cuál sería la característica estructural de los elementos que corresponden a cada celda no nula de la planilla, suponiendo que puede almacenar números enteros, fórmulas o textos como cadenas?

Solución:

Representación: Matriz Dispersa (Sparse Matrix)

Para una planilla de 999×999 celdas donde la mayoría están vacías, se utiliza una **matriz dispersa** que almacena únicamente las celdas no nulas mediante listas enlazadas bidimensionales.

Estructura del Nodo de Celda

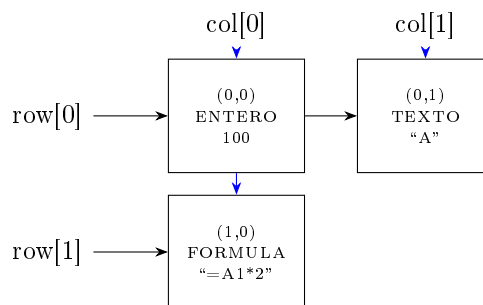
Cada celda no nula se representa con un nodo que contiene:

```
1 class CellNode {
2     // Coordenadas
3     int row;           // 0 a 998
4     int column;        // 0 a 998
5
6     // Tipo y contenido
7     DataType type;     // ENTERO | FORMULA | TEXTO
8     Object value;      // Numero entero, formula o texto
9
10    // Punteros bidimensionales
11    CellNode nextRow;   // Siguiete celda en la misma fila
12    CellNode nextCol;   // Siguiete celda en la misma columna
13 }
14
15 enum DataType {
16     ENTERO,           // Numero entero
17     FORMULA,          // Formula (cadena que empieza con '=')
18     TEXTO              // Cadena de texto
19 }
```

Características Estructurales

1. **Dos punteros por celda:** `nextRow` y `nextCol` permiten navegación eficiente tanto horizontal (por filas) como vertical (por columnas).
2. **Arrays de cabeceras:**
 - `rowHeaders[999]`: Apunta al primer nodo no nulo de cada fila
 - `colHeaders[999]`: Apunta al primer nodo no nulo de cada columna
3. **Tipo polimórfico:** El campo `value` almacena diferentes tipos de datos según `type`:
 - **ENTERO**: Un número entero
 - **FORMULA**: Cadena representando la fórmula (ej: “=A1+B2”)
 - **TEXTO**: Cadena de caracteres

Visualización de la Estructura



Ventajas

- **Eficiencia espacial:** Solo almacena las k celdas no nulas, en lugar de $999 \times 999 = 998,001$ espacios.
- **Navegación eficiente:** Recorrido rápido por filas o columnas específicas.
- **Inserción/eliminación eficiente:** Solo requiere ajustar punteros.

Ejemplo: Para una planilla con solo 3 celdas ocupadas, la matriz dispersa usa $1,998$ cabeceras $+3$ nodos $= 2,001$ elementos, comparado con $998,001$ en una matriz completa.

Ahorro: 99.8 % de memoria.

[↑ Volver al índice](#)